Lab 4. Working with Pods, Deployments and Service

In this lab, you will create a Container Registry and will also perform some operations such as push and pull a Docker image and run application on Kubernetes.

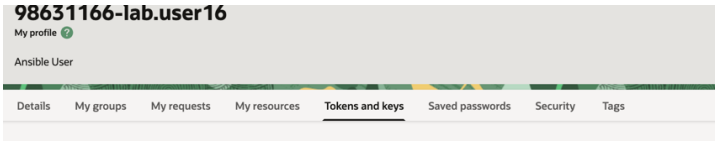
1. Create an Auth Token.
2. Create a new Container Repository.
3. Sign in to Oracle Cloud Infrastructure Registry (OCIR) from the Cloud Shell.
4. Tag the Docker image.
5. Push the tagged Docker image to OCIR Repository.
6. Verify if the image has been pushed.
7. Pull the image from OCIR Repository.
8. Create and Manage Pods
9. Working with Deployment
10. Working with Service

Pre Requisite:

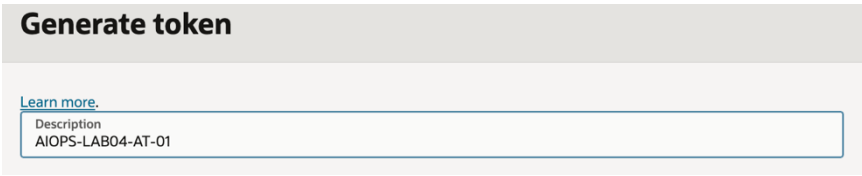
- A Working Kubernetes Cluster.

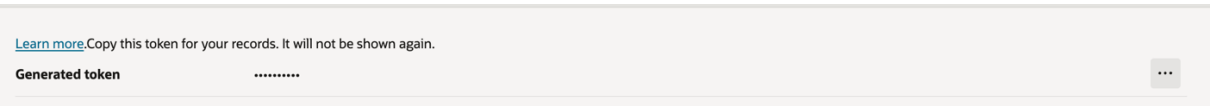
Sequence 1. Create an Auth Token for OCIR

1. Login to Oracle Cloud with give credentials.
2. In the top-right corner of the OCI Console, open the **Profile** menu, and then click **User Settings**.



3. Click on **Tokens and Keys** from the top navigation menu, then click **Generate Token**. Enter **AIOPS-LAB04-AT-01**, as description for the auth token.
4. Click **Generate Token** button from bottom right corner..





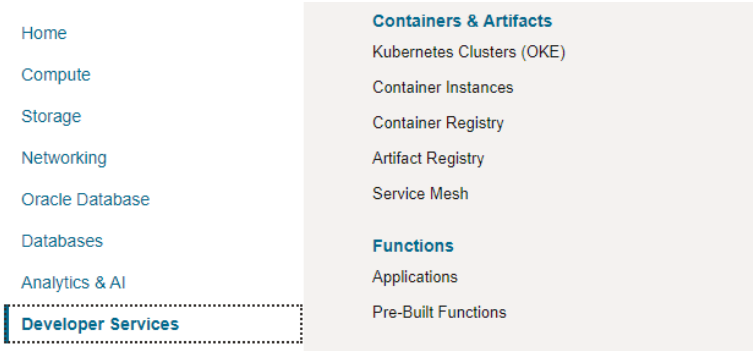
Copy the auth token by right clicking on three dots at the end of the row to a notepad because you won't see the auth token again in the Console. You'll need this auth token as password later in this and other labs.

Sequence 2. Create a New Container Repository on OCIR

Create an empty repository in a compartment and give it a name that's unique across all compartments in the tenancy. Having created the new repository, you can push an image to

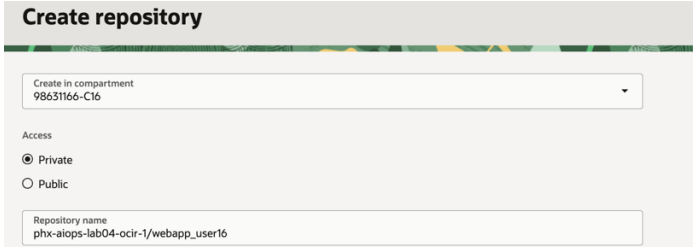
the repository using the Docker CLI.

1. Check if you can access Oracle Cloud Infrastructure Registry (OCIR):
   a. In the Console, open the navigation menu and click **Developer Services.** Under **Containers & Artifacts**, click **Container Registry**.



   b. Select your *<Compartment Name>* from List scope on the left menu.
2. Click **Create Repository**.
3. Select your *<Compartment Name>* to create a new repository.
4. Enter the name: *<region-key>*-dop-lab04-ocir/webapp_*<userID>*

   For example, `iad-aiops-lab04-ocir-1/webapp_user16`

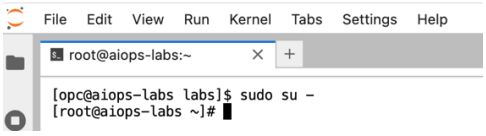


5. Select the **Private** option to limit access to the new repository.
6. Click **Create Repository**.

Sequence 3. Sign In to OCIR from your Instance and Tag your Image.

Once you have generated the auth token and created a new repository, sign to OCIR from your **Instance.**

1. Open the Jupyter Lab server, launch terminal and switch to root user.



2. Log in to OCIR by entering:

   ```
   podman login iad.ocir.io
   ```

3. When prompted for a username, enter your username in the format `<tenancy-namespace>/<username>`

For example, `ocuocictrng24/98631166-lab.user16`

Enter the auth token (random string) you copied earlier as the password.

4. Clone a demo repository from Github.

```
git clone https://github.com/ou-developers/docker-helloworld-demo

cd docker-helloworld-demo/
```

5. Inspect the files you have in the directory including application code and Dockerfile for creating the sample Nginx application.
6. Create a container image for the sample Web Application using the `podman build` command. Select Docker.io. Replace <User_d> with your user ID.

```
podman build -t webapp_<user_id>:1.0 .

E.g.: podman build -t webapp_user16:1.0 .
```

```
[root@aiops-labs docker-helloworld-demo]# podman build -t webapp_user16:1.0 .
STEP 1/11: FROM iad.ocir.io/ocuocictrng5/nginx:latest
Trying to pull iad.ocir.io/ocuocictrng5/nginx:latest...
```

```
--> f1bbc1b1b9f3
Successfully tagged localhost/webapp_user16:1.0
f1bbc1b1b9f38eb65e31e6c5c8ad4d3801d408817fc9937769c18611c5e0d3b8
[root@aiops-labs docker-helloworld-demo]#
```

7. Verify the image in the local repository using the following command:

```
podman images
```

8. Run the following command to attach a tag to the image that you're going to push to OCIR repository:

```
podman tag webapp_<user id>:1.0 <region-key>.ocir.io/<tenancy-namespace>/<repo-name>:<tag>
```

- `<region-key>` is the key for the OCIR region you're using. For example, `phx` is the region key for `US West` (Phoenix) region.
- *`<tenancy-namespace>`* is the Object Storage namespace string of the tenancy, for example, `ocuocictrng24`.
- *`<repo-name>`* is the name of the target repository to which you want to push the image (`iad-aiops-lab04-ocir-1/webapp_user16`).
- *`<tag>`* is an image tag you want to give the image in OCIR (for example, latest).

For example,

```
podman tag webapp_user16:1.0 \
iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:latest
```

9. Validate if the new image with the tag is listed.

```
podman images
```

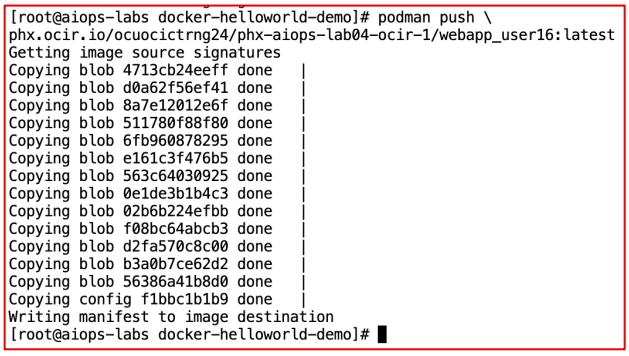
Sequence 4. Push the Tagged Image to OCIR Repository and Verify

1. Run the following command to push the tagged Container image to OCIR repository:

   `podman push <region-key>.ocir.io/<tenancy-namespace>/<repo-name>:<tag>`

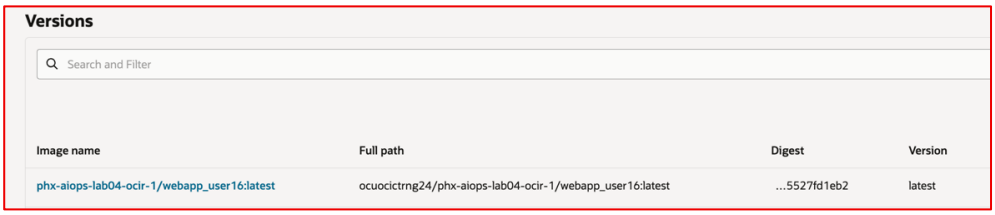
   For example,

   ```
   podman push \
   iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:latest
   ```



```
[root@aiops-labs docker-helloworld-demo]# podman push \
phx.ocir.io/ocuocictrng24/phx-aiops-lab04-ocir-1/webapp_user16:latest
Getting image source signatures
Copying blob 4713cb24eeff done   |
Copying blob d0a62f56ef41 done   |
Copying blob 8a7e12012e6f done   |
Copying blob 511780f88f80 done   |
Copying blob 6fb960878295 done   |
Copying blob e161c3f476b5 done   |
Copying blob 563c64030925 done   |
Copying blob 0e1de3b1b4c3 done   |
Copying blob 02b6b224efbb done   |
Copying blob f08bc64abcb3 done   |
Copying blob d2fa570c8c00 done   |
Copying blob b3a0b7ce62d2 done   |
Copying blob 56386a41b8d0 done   |
Copying config f1bbc1b1b9 done   |
Writing manifest to image destination
[root@aiops-labs docker-helloworld-demo]#
```

2. Go back to the OCIR page and select your `<Compartment Name>` from List scope.
3. You'll see the repository `iad-aiops-lab04-ocir-1/webapp_<userID>`.
4. Click the name of the repository that contains the image you just pushed. You'll see:



Versions

Search and Filter

Image name	Full path	Digest	Version
phx-aiops-lab04-ocir-1/webapp_user16:latest	ocuocictrng24/phx-aiops-lab04-ocir-1/webapp_user16:latest	...5527fd1eb2	latest

- An image with the tag `latest`. A summary page that shows you the details about the repository, including who created it and when, its size, and whether it's a public or a private repository.

Sequence 5. Pull the Image from OCIR Repository

1. Delete the existing images from the local repository. Run `podman rmi` command to delete your newly created images.

   `podman rmi <Image ID> --force. For Example`

   ```
   podman rmi f1bbc1b1b9f3 --force
   ```

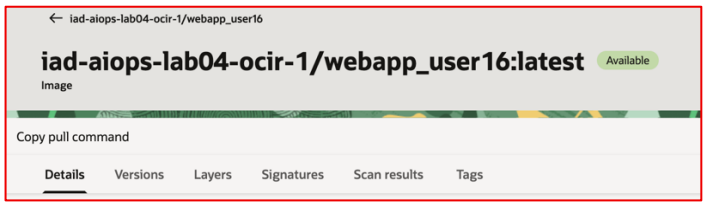
2. Verify if the images are deleted.

   ```
   podman images
   ```

3. Switch to the OCI Console. From the OCIR page, select the repository and the image tag that

needs to be pulled.

4. Click on **Copy pull command**.



5. The command you copy includes the fully qualified path to the image's location. Paste the command in notepad and replace **docker** with **podman**.
6. Execute the copied command in terminal.

```
podman pull iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-1/webapp_user16:latest
```

7. Verify the pulled image from OCIR repository.

```
podman images
```

8. Exit from the Instance but stay in Cloud Shell.

```
exit
```

Sequence 6. Create a Kubernetes (OKE) Secret

To enable Kubernetes to pull an image from OCIR repository when deploying an application, you need to create a Kubernetes secret. The secret contains all the login details you would provide while logging in to OCIR using the `docker login` command, including your auth token.

1. Open the terminal of jupyter lab.
2. Run the following command to create a secret:

```
kubectl create secret docker-registry ocir-secret-<userID> \
--docker-server=iad.ocir.io \
--docker-username='ocuocictrng24/98631166-lab.user16' \
--docker-password='<oci-auth-token>' \
--docker-email='<userID>@acme.com'
```

Where,

- *<name-of-secret>-<userID>*: A unique name for the secret.
- `ocir.io` is the Oracle Cloud Infrastructure Registry name.
- *<tenancy-namespace>* is the Object Storage namespace string of the tenancy.
- <oci-username> provide the full username as given in **the profile menu**
- *<oci-auth-token>*: Use the auth token (random string) created earlier.

```
[root@aiops-labs docker-helloworld-demo]# kubectl create secret docker-registry ocir-secret-user16 \
--docker-server=phx.ocir.io \
--docker-username='ocuocictrng24/98631166-lab.user16' \
--docker-password=':f1:}h:xgpUD2HcNo>t;' \
--docker-email='user16@acme.com'
secret/ocir-secret-user16 created
[root@aiops-labs docker-helloworld-demo]#
```

3. Run the following command to verify if the secret has been created:

```
kubectl get secrets
```

```
[root@aiops-labs docker-helloworld-demo]# kubectl get secrets
NAME                  TYPE                                  DATA   AGE
default-token-l5rhm   kubernetes.io/service-account-token   3      10h
ocir-secret-user16    kubernetes.io/dockerconfigjson        1      35s
[root@aiops-labs docker-helloworld-demo]#
```

Sequence 7. Create and Manage a Pod

1. Launch the terminal from jupyter lab and switch to root user.
2. Create Pod manifest file, pod.yaml:

```
vi pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    zone: prod
    version: v1
spec:
  containers:
    - name: hello-ctr
      image: iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-
1/webapp_user16:latest
      ports:
        - containerPort: 8080
  imagePullSecrets:
    - name: ocir-secret-user16
```

3. Use the following kubectl command to Deploying Pods from a manifest file to the API server.

```
kubectl apply -f pod.yaml
```

4. Run a kubectl get pods command to check the status.

```
kubectl get pods
```

```
[root@aiops-labs ~]# kubectl apply -f pod.yaml
pod/hello-pod created
[root@aiops-labs ~]# vi pod.yaml
[root@aiops-labs ~]# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
hello-pod   1/1     Running   0          58s
[root@aiops-labs ~]#
```

5. Get the full version of the pod deployment with kubectl get pods -o yaml command.

```
kubectl get pods hello-pod -o yaml
```

6. Use kubectl describe for a nicely formatted multi-line overview of an object.

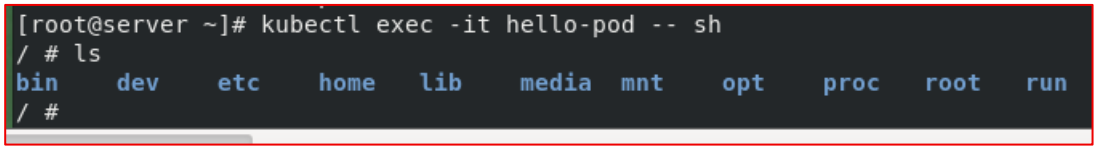
```
kubectl describe pods hello-pod
```

7. Introspect a running Pod. Log into it or execute commands in it. We can do both of these with the kubectl exec command.

```
kubectl exec hello-pod -- ls
```

8. Log-in to containers running in Pods using kubectl exec. When you do this, your terminal prompt will change to indicate your session is now running inside of a container in the Pod, and you'll be able to execute commands from there.

```
kubectl exec -it hello-pod -- sh
```



9. Delete your pod typing exit to quit your shell session inside the container, then run kubectl delete to delete the Pod.

```
# exit
kubectl delete -f pod.yaml
```

Sequence 8. Working with Deployment

1. Create a deployment from a YAML file, deploy.yml

```
vi deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-deploy
spec:
  replicas: 5
  selector:
    matchLabels:
      app: hello-world
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-pod
        image: iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-
1/webapp_user16:latest
        ports:
        - containerPort: 8080
      imagePullSecrets:
        - name: ocir-secret-user16
```

2. Use kubectl apply to implement it on the cluster.

```
kubectl apply -f deploy.yml
```

3. Check the status of deployment.

```
kubectl get deployments
```

4. Use `kubectl delete` to delete the deployment.

```
kubectl delete -f deploy.yml
```

Sequence 9. Working with Service

1. Use kubectl to declaratively deploy the YAML file, deploy-service.yml.

```
# vi deploy-service.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: svc-test
spec:
  replicas: 5
  selector:
    matchLabels:
      chapter: services
  template:
    metadata:
      labels:
        chapter: services
    spec:
      containers:
      - name: hello-ctr
        image: iad.ocir.io/ocuocictrng24/iad-aiops-lab04-ocir-
1/oci_webapp_user16:latest
        ports:
        - containerPort: 8080
```

2. Deploy the manifest on the cluster

```
kubectl apply -f deploy-service.yml
```

3. Use the following kubectl command to create a new Service that will provide networking through NodePort for the Pods deployed in the previous step.

```
kubectl expose deployment svc-test \
--name=hello-svc \
--target-port=80 \
--type=NodePort
```

   - *The* `kubectl expose` *is the imperative way to create a new Service object.*
   - `deployment svc-test` *is telling Kubernetes to expose the svc-test Deployment that we created in the previous step.*
   - `--name=hello-svc` *tells Kubernetes that we want to call this Service "hello-svc".*
   - `--target-port=8080` *tells it which port the app is listening on.*
   - `--type=NodePort` *tells Kubernetes we want a cluster-wide port for the Service.*
4. Inspect the service with the `kubectl describe svc hello-svc` command.

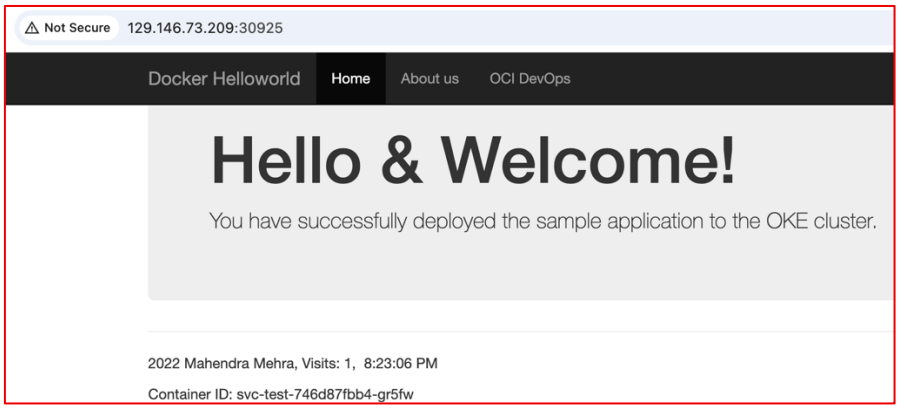
```
kubectl describe svc hello-svc
```

```
[root@aiops-labs ~]# kubectl describe svc hello-svc
Name:                     hello-svc
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 chapter=services
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.43.92.107
IPs:                      10.43.92.107
Port:                     <unset>  8080/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30925/TCP
Endpoints:                10.42.0.17:80,10.42.0.18:80,10.42.0.19:80 + 2 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Events:                   <none>
[root@aiops-labs ~]#
```

Some interesting values in the output include:

- *Selector is the list of labels that Pods must have in order for the Service to send traffic to them*
- *IP is the permanent ClusterIP (VIP) of the Service*
- *Port is the port that the app and Service listens on*
- *NodePort is the cluster-wide port that can be used to access it from outside the cluster*

5. Use the Public IP address of the Instance on the ***NodePort 30925***



6. Clean up by deleting the Service and deployment we just created.

```
kubectl delete svc hello-svc
kubectl delete -f deploy-service.yml
```

7. **Congratulations! You have successfully completed this Lab.**